CHR

Software Design Document

High-Level Design (HLD) — CHR-System v2.0

C4 Architecture	Data Flow	DB Schema	API Contracts	Security Model	Deployment

Document ID	CHR-SDD-HLD-001	Version	1.0
Author	Madhavan Shivakumar	Status	DRAFT → APPROVED
Date	June 27, 2026	Classification	Internal / Confidential
Repository	github.com/Madhavan-dev18/chr-system	Review Cycle	Per sprint

This document defines the software architecture, system components, data models, interface contracts, security design, and deployment topology for CHR-System — an open-source, HIPAA-aligned Electronic Health Records platform built on Next.js 15, PostgreSQL, and a fully containerised open-source stack. It serves as the primary technical reference for all engineering decisions.

Table of Contents

01. Introduction & Scope

1.1 Purpose

This Software Design Document (SDD) provides a complete high-level design specification for the CHR-System. It translates the requirements captured in the Product Requirements Document (PRD-v2.0) into concrete architectural decisions, component boundaries, interface contracts, and implementation guidelines. All engineering work on CHR-System must reference this document for design intent.

1.2 Scope

- Full-stack web application: Next.js 15 frontend + API layer
- PostgreSQL 16 database with Prisma ORM and Row-Level Security
- Authentication and RBAC system (5 roles)
- Medical records management with AES-256-GCM encryption at rest
- Appointment scheduling engine with async notification dispatch
- AI-powered clinical decision support (Ollama, HuggingFace)
- Observability stack: Prometheus, Grafana, Sentry, OpenTelemetry
- CI/CD pipeline: GitHub Actions with Docker Compose
- Zero-cost, open-source infrastructure only

1.3 Definitions & Abbreviations

Term	Definition
SDD	Software Design Document
HLD	High-Level Design
EHR	Electronic Health Record
MRN	Medical Record Number — unique patient identifier
RBAC	Role-Based Access Control
PHI	Protected Health Information — HIPAA-regulated patient data
tRPC	TypeScript Remote Procedure Call — end-to-end type-safe API layer
RLS	Row-Level Security — PostgreSQL access policy per row
BullMQ	Bull Message Queue — Redis-backed async job processing
ISR	Incremental Static Regeneration — Next.js caching strategy
ADR	Architecture Decision Record
OWASP	Open Web Application Security Project
JWT	JSON Web Token
FHIR	Fast Healthcare Interoperability Resources — HL7 standard

02. System Overview

2.1 System Description

CHR-System is a multi-role, HIPAA-aligned Clinical Health Records platform designed for small-to-medium healthcare facilities. It provides patient registration, appointment scheduling, medical record management, vitals monitoring, prescription tracking, lab result management, AI-assisted clinical decision support, and a complete HIPAA audit trail — all on a zero-cost, fully open-source infrastructure.

2.2 Design Principles

Principle	Application in CHR-System
Separation of Concerns	Frontend, API, business logic, and data layers are strictly decoupled
Security by Default	Every endpoint authenticated; RBAC checked before any data access
Privacy by Design	PHI encrypted at application layer; audit log on every access
Fail-Safe Defaults	Deny access unless explicitly granted; no implicit permissions
Zero Trust	Each service authenticates independently; no implicit internal trust
Defence in Depth	Multiple security layers: WAF → rate limit → auth → RBAC → RLS → encryption
Loose Coupling	Services communicate through well-defined contracts; replaceable components
High Cohesion	Each module owns a single domain (patients, appointments, records)
Observability First	Every service emits structured logs, metrics, and distributed traces
Open Source / No Lock-in	All infrastructure replaceable; no proprietary services required

2.3 Quality Attributes (Non-Functional Requirements)

Attribute	Target	Mechanism
Availability	99.5% uptime	Railway auto-restart + health checks
Performance	p95 API < 150ms	Redis cache + PgBouncer + indexed queries
Scalability	1,000 concurrent users	Stateless Next.js + connection pool
Security	OWASP Top 10 clean	ZAP scan in CI; penetration test checklist
Maintainability	>80% test coverage	Vitest + Playwright; enforced in CI
Auditability	100% PHI access logged	Append-only audit_logs table; no deletes
Recoverability	RTO < 1h, RPO < 24h	PostgreSQL WAL + nightly automated backup
Accessibility	WCAG 2.1 AA	axe-core in Playwright E2E; aria labels
Portability	Docker single-command	docker compose up — full stack in one step

03. Architecture Design — C4 Model

3.1 Architectural Pattern

CHR-System follows a Layered Architecture with elements of Domain-Driven Design. The top level is a monorepo containing the web application and shared packages. Within the app, layers are: Presentation (React components) → Application (tRPC procedures) → Domain (business logic services) → Infrastructure (Prisma, Redis, MinIO). This enables clear separation while keeping the codebase navigable for a solo developer or small team.

Figure 1 — C4 Context Diagram

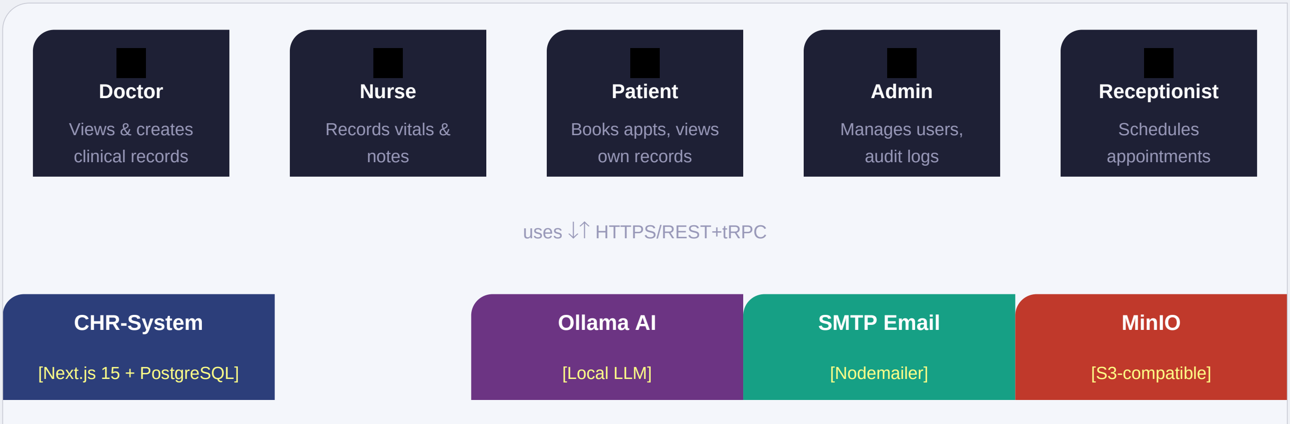

Figure 1 · CHR-System C4 Context — actors and external systems

Figure 2 — C4 Container Diagram

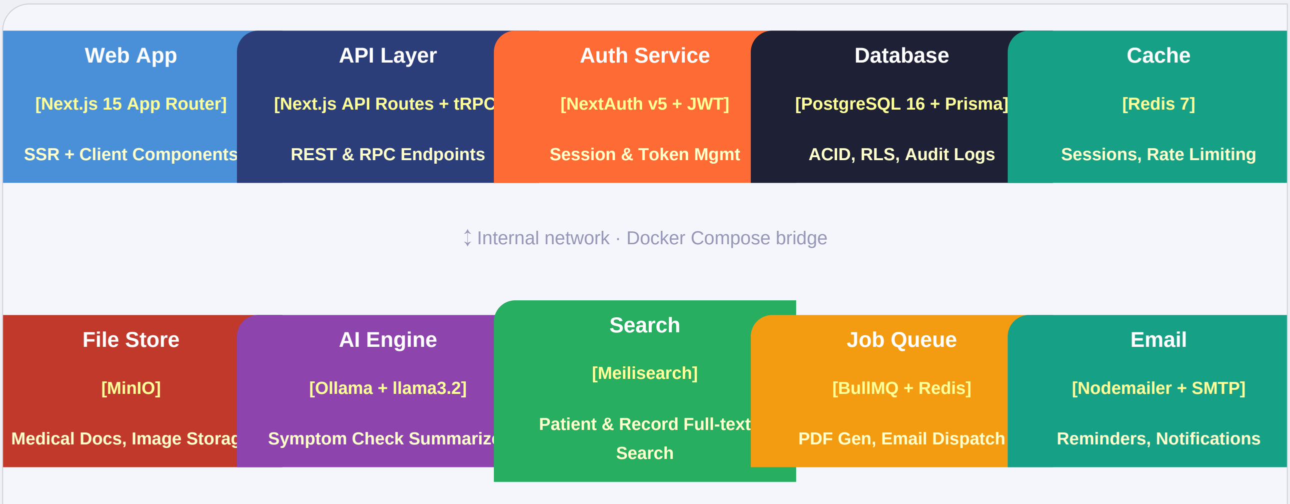

Figure 2 · C4 Container — all runtime services and their responsibilities

3.2 Module Decomposition

Module	Responsibility	Key Files
auth	Session management, JWT issuance/validation, MFA	lib/auth.ts, api/auth/*

patients	Patient CRUD, MRN generation, demographic management	app/patients/*, trpc/patients.ts
appointments	Scheduling, calendar, conflict detection, reminders	app/appointments/*, workers/reminder.ts
medical-records	EHR CRUD, encryption wrapper, version history	lib/crypto.ts, trpc/records.ts
vitals	Time-series vitals recording, baseline comparison	trpc/vitals.ts, components/charts/*
prescriptions	Medication management, drug interaction check	trpc/prescriptions.ts
lab-results	Lab CRUD, reference range alerts, trend analysis	trpc/labs.ts
ai-engine	Ollama adapter, symptom checker, summarizer, ICD-10	lib/ai/ollama.ts, workers/ai.ts
audit	Append-only PHI access log, admin viewer, CSV export	lib/audit.ts, app/admin/audit/*
notifications	BullMQ job dispatch, email/SMS/in-app fan-out	workers/notify.ts, lib/email.ts
search	Meilisearch sync, patient/record full-text search	lib/search.ts, workers/indexer.ts
reports	Async PDF generation, ReportLab pipeline, MinIO save	workers/pdf.ts, lib/pdf.ts
billing	Invoice creation, insurance claim tracking (Phase 4)	trpc/billing.ts (future)
admin	User management, clinic settings, audit dashboard	app/admin/*

04. Component Design

4.1 Frontend Component Architecture

The frontend uses Next.js 15 App Router with a clear component hierarchy. Server Components handle data fetching and layout. Client Components handle interactivity. Shared UI components are built on shadcn/ui with neumorphic design tokens overriding defaults.

Layer	Type	Responsibility	Example Components
Page Layer	Server Component	Data fetch via tRPC server-side caller	app/patients/page.tsx
Layout Layer	Server Component	Auth guard, sidebar, top bar	app/(dashboard)/layout.tsx
Feature Components	Client Component	Domain-specific UI with state	PatientCard, VitalsChart
UI Primitives	Client Component	shadcn/ui + neumorphic overrides	NeuCard, NeuBtn, NeuInput
Form Components	Client Component	Zod-validated react-hook-form wrappers	PatientForm, VitalEntryForm
Chart Components	Client Component	Recharts wrappers for health data	SparkLine, DonutChart, GaugeRing
Provider Layer	Client Component	tRPC, NextAuth, React Query providers	app/providers.tsx

4.2 API Layer — tRPC Routers

The tRPC layer sits between React components and the database. Each router corresponds to a domain module. All procedures are protected by a middleware chain: auth check → role check → input validation → audit log.

Router	Procedures (key)	Auth Required
authRouter	login, logout, refresh, changePassword, enableMFA	Mixed (public login)
patientRouter	list, getById, create, update, search, delete	DOCTOR/NURSE/ADMIN
appointmentRouter	list, book, reschedule, cancel, getAvailability	All roles (scoped)
recordRouter	list, create, update, getEncrypted, generatePDF	DOCTOR only (create)
vitalsRouter	record, getTimeSeries, getLatest, getBaseline	DOCTOR/NURSE
prescriptionRouter	list, create, discontinue, checkInteractions	DOCTOR
labRouter	list, create, flagAbnormal, getTrend	DOCTOR/LAB_TECH
aiRouter	symptomCheck, summarize, suggestICD10, generateDischarge	DOCTOR
auditRouter	list, exportCSV, getByUser, getByResource	ADMIN only
adminRouter	listUsers, updateRole, deactivate, getClinicStats	ADMIN only
notifyRouter	getInbox, markRead, updatePreferences	All auth users

4.3 Middleware Chain

Every tRPC procedure passes through this ordered middleware chain:

Step	Description	Layer
1. JWT Verification	Decode & verify access token; reject if expired/invalid	Middleware
2. Session Hydration	Load user record from Redis session cache or DB	Middleware
3. RBAC Check	Verify user role ∈ allowed roles for this procedure	Middleware
4. Clinic Scope	Filter query context to user's clinic_id (multi-tenancy)	Middleware
5. Input Validation	Run Zod schema; reject malformed or over-length payloads	Middleware
6. Rate Limit Check	Redis sliding window; 429 if threshold exceeded	Middleware
7. Procedure Execution	Business logic runs within validated, authorised context	Middleware
8. Audit Log Write	Append-only INSERT to audit_logs (async, non-blocking)	Middleware
9. Response Sanitise	Strip internal fields; return typed response via Zod output	Middleware

05. Database Design

5.1 Entity Relationship Overview

PostgreSQL 16 with Prisma ORM. UUID v7 primary keys throughout. Soft deletes (deleted_at) on all clinical tables. Row-Level Security enforced at database layer as a secondary defence beyond application RBAC. The schema is normalised to 3NF with denormalisation only where performance profiling justifies it.

5.2 Entity Specifications

Table: users

`PK: id UUID v7 PRIMARY KEY`

Column	Type	Constraint / Notes
email	VARCHAR(320)	UNIQUE NOT NULL
password_hash	TEXT	bcrypt hash — never plaintext
role	ENUM	ADMIN|DOCTOR|NURSE|PATIENT|RECEPTIONIST|LAB_TECH
clinic_id	UUID	FK → clinics.id — multi-tenancy scope
is_active	BOOLEAN	Soft disable without deleting record
mfa_secret	TEXT	TOTP secret — AES encrypted at rest
last_login_at	TIMESTAMPTZ	Track account activity
failed_logins	INTEGER	Increment on fail; lock at 5
locked_until	TIMESTAMPTZ	NULL = not locked; rate-limit lockout
created_at	TIMESTAMPTZ	DEFAULT NOW()
deleted_at	TIMESTAMPTZ	Soft delete — NULL = active

Table: patients

`PK: id UUID v7 PRIMARY KEY`

Column	Type	Constraint / Notes
mrn	VARCHAR(20)	UNIQUE — auto-generated MRN-YYYYNNNNN
first_name	VARCHAR(100)	NOT NULL
last_name	VARCHAR(100)	NOT NULL
dob	DATE	Date of birth
gender	ENUM	MALE|FEMALE|OTHER|PREFER_NOT_TO_SAY
blood_type	ENUM	A+|A-|B+|B-|O+|O-|AB+|AB-|UNKNOWN
allergies	TEXT[]	Array of known allergen strings

phone	VARCHAR(20)	E.164 format
email	VARCHAR(320)	Optional — for patient portal
address	JSONB	{street, city, state, pin, country}
emergency_contact	JSONB	{name, relation, phone}
clinic_id	UUID	FK → clinics.id
user_id	UUID	FK → users.id — if patient has portal access
deleted_at	TIMESTAMPTZ	Soft delete

Table: medical_records

`PK: id UUID v7 PRIMARY KEY`

Column	Type	Constraint / Notes
patient_id	UUID	FK → patients.id NOT NULL
doctor_id	UUID	FK → users.id (role=DOCTOR) NOT NULL
record_type	ENUM	CONSULTATION|PROCEDURE|EMERGENCY|DISCHARGE|LAB_ORDER
encrypted_content	BYTEA	AES-256-GCM ciphertext of clinical notes JSON
iv	BYTEA	AES initialisation vector (12 bytes)
auth_tag	BYTEA	AES-GCM authentication tag (16 bytes)
diagnosis_codes	VARCHAR(10)[]	ICD-10 codes array
attachments	JSONB	[{filename, minio_key, size, mime_type}]
version	INTEGER	Optimistic concurrency — increments on update
clinic_id	UUID	FK → clinics.id
created_at	TIMESTAMPTZ	DEFAULT NOW()
deleted_at	TIMESTAMPTZ	Soft delete — audit preserved even after

Table: audit_logs

`PK: id BIGSERIAL PRIMARY KEY`

Column	Type	Constraint / Notes
user_id	UUID	FK → users.id — who performed the action
action	ENUM	VIEW|CREATE|UPDATE|DELETE|EXPORT|LOGIN|LOGIN_FAIL|LOGOUT
resource	VARCHAR(50)	Table / feature accessed
resource_id	UUID	Specific record accessed (nullable)
clinic_id	UUID	FK → clinics.id
ip_address	INET	Client IP
user_agent	TEXT	Browser/client user-agent string

request_id	UUID	Distributed trace correlation
metadata	JSONB	Additional context (query params, diff)
timestamp	TIMESTAMPTZ	DEFAULT NOW() — indexed for range queries

5.3 Indexes

Index Name	Table	Columns	Type
idx_users_email	users	email	B-tree UNIQUE
idx_patients_mrn	patients	mrn	B-tree UNIQUE
idx_patients_clinic	patients	clinic_id, deleted_at	B-tree composite
idx_appointments_date	appointments	doctor_id, scheduled_at	B-tree composite
idx_records_patient	medical_records	patient_id, created_at DESC	B-tree composite
idx_vitals_patient_time	vitals	patient_id, recorded_at DESC	B-tree composite
idx_audit_logs_timestamp	audit_logs	timestamp DESC	B-tree BRIN
idx_audit_logs_user	audit_logs	user_id, timestamp	B-tree composite
idx_prescriptions_patient	prescriptions	patient_id, is_active	B-tree composite
idx_records_fts	medical_records	diagnosis_codes	GIN (array ops)

06. API Design — Interface Contracts

6.1 API Architecture

CHR-System uses a hybrid API strategy: tRPC for type-safe internal Next.js client-server communication, and REST endpoints for third-party integration and webhook receivers. Both live under /api/. REST endpoints are additionally documented via auto-generated Swagger UI.

6.2 REST Endpoint Contracts

Method	Path	Request Body / Params	Response	Auth
POST	/api/auth/login	{email, password}	{accessToken, refreshToken}	Public
POST	/api/auth/refresh	{refreshToken} (cookie)	{accessToken}	Refresh JWT
POST	/api/auth/logout	–	{ok: true}	JWT
GET	/api/patients	?page&limit&search&status	Paginated<Patient[]>	DOCTOR+
POST	/api/patients	PatientCreateInput	Patient	ADMIN/RECEPT
GET	/api/patients/:id	–	PatientWithRelations	DOCTOR/NURSE
PATCH	/api/patients/:id	Partial<PatientUpdateInput>	Patient	DOCTOR/ADMIN
GET	/api/patients/:id/records	?type&from&to	MedicalRecord[]	DOCTOR (audit)
POST	/api/patients/:id/records	RecordCreateInput (encrypted)	MedicalRecord	DOCTOR
GET	/api/appointments	?date&doctorId&status	Appointment[]	Role-scoped
POST	/api/appointments	AppointmentCreateInput	Appointment	Any auth
PATCH	/api/appointments/:id	{status, notes}	Appointment	DOCTOR/ADMIN
GET	/api/vitals/:patientId	?from&to&type	VitalReading[]	DOCTOR/NURSE
POST	/api/vitals	VitalsCreateInput	VitalReading	NURSE/DOCTOR
GET	/api/audit-logs	?userId&from&to&action	Paginated<AuditLog[]>	ADMIN only
GET	/api/reports/:patientId	?format=pdf	{ jobId }	DOCTOR/ADMIN
GET	/api/ai/symptom-check	?symptoms&patientId	DifferentialDiagnosis[]	DOCTOR
POST	/api/webhooks/payment	Stripe/Razorpay payload	{ received: true }	Webhook sig

6.3 Standard Response Envelope

```
// Success
{ "data": { ... }, "meta": { "requestId": "uuid", "timestamp": "ISO" } }
// Error
{ "error": { "code": "UNAUTHORIZED", "message": "...", "field": "email" } }
```

6.4 Pagination Contract

```
// Cursor-based (keyset) pagination — stable under concurrent inserts
GET /api/patients?cursor=<uuid>&limit;=20&sort;=createdAt■=desc
Response: { "data": [...], "nextCursor": "<uuid>|null", "total": 1284 }
```

6.5 Error Codes

Code	HTTP Status	Description
UNAUTHORIZED	401	Missing or invalid JWT
FORBIDDEN	403	Role not permitted for this resource
NOT_FOUND	404	Resource does not exist or deleted
VALIDATION_ERROR	422	Zod schema validation failure — includes field
RATE_LIMITED	429	Too many requests — Retry-After header set
INTERNAL_ERROR	500	Unexpected server error — requestId logged
RECORD_LOCKED	409	Optimistic concurrency conflict — version mismatch
CLINIC_SCOPE_VIOLATION	403	Attempt to access cross-clinic data — security event

07. Authentication & Authorization Design

7.1 Authentication Flow

Step	Action	Implementation
1	User submits email + password	POST /api/auth/login
2	Rate limit check: 5 failures → 15min lock	Redis INCR + TTL on key auth:fail:{email}
3	Fetch user by email	Prisma: users.findUnique({where: {email}})
4	Verify password hash	bcrypt.compare(input, user.password_hash)
5	If MFA enabled: verify TOTP	speakeasy.totp.verify(secret, token)
6	Issue access token (15min TTL)	jwt.sign({sub, role, clinicId}, ACCESS_SECRET)
7	Issue refresh token (7 day TTL)	crypto.randomBytes(32) — store bcrypt hash in Redis
8	Set refresh token as HttpOnly SameSite cookie	res.cookie('rt', token, {httpOnly, secure, sameSite})
9	Log successful login in audit_logs	audit.log({action: 'LOGIN', userId, ip})
10	Return access token in JSON response body	{ accessToken } — stored in memory, not localStorage

7.2 RBAC Permission Matrix

Resource / Action	ADMIN	DOCTOR	NURSE	PATIENT	RECEPTIONIST	LAB_TECH
List all patients	✓	✓	✓	✗	✓	✗
View own record only	—	—	—	✓	—	—
Create medical record	✗	✓	✗	✗	✗	✗
Record vitals	✗	✓	✓	✗	✗	✗
Create prescription	✗	✓	✗	✗	✗	✗
Book appointment	✓	✓	✓	✓	✓	✗
Cancel appointment	✓	✓	✗	✓ (own)	✓	✗
Enter lab results	✗	✗	✗	✗	✗	✓
View audit logs	✓	✗	✗	✗	✗	✗
Manage users/roles	✓	✗	✗	✗	✗	✗
Export patient list	✓	✗	✗	✗	✗	✗
Access AI engine	✗	✓	✗	✗	✗	✗
Generate PDF report	✓	✓	✗	✗	✗	✗

08. Security Architecture

8.1 Encryption Design

Data Type	Encryption	Key Management	Notes
Medical record content	AES-256-GCM	App-level key in env var	IV + auth tag stored per record
Database passwords	bcrypt (cost=12)	Hash only — no plaintext ever	Min 8 chars + complexity enforced
Refresh tokens	bcrypt hash in Redis	Random 256-bit before hashing	Single-use; rotate on each refresh
MFA secrets	AES-256-CBC	Separate MFA_SECRET env var	TOTP: RFC 6238 compliant
Data in transit	TLS 1.3 (minimum)	Cloudflare / Let's Encrypt	HSTS header with 1-year max-age
MinIO objects	SSE-C (client-side enc)	Per-object key derived from MRN	Files encrypted before S3 write
DB backups	GPG symmetric encryption	Backup passphrase in vault	Backup key rotated quarterly

8.2 Security Headers

Header	Value	Protects Against
Content-Security-Policy	default-src 'self'; script-src 'self' 'nonce-{n}'	XSS injection
Strict-Transport-Security	max-age=31536000; includeSubDomains; preload	SSL stripping, MITM
X-Frame-Options	DENY	Clickjacking
X-Content-Type-Options	nosniff	MIME sniffing
Referrer-Policy	strict-origin-when-cross-origin	Referrer leakage
Permissions-Policy	camera=(), microphone=(), geolocation=()	Feature abuse
Cross-Origin-Opener-Policy	same-origin	Cross-origin leaks

8.3 Threat Model Summary

Threat	Likelihood	Impact	Control
SQL Injection	Low	Critical	Prisma parameterised queries; no raw SQL
XSS	Medium	High	CSP nonce; DOMPurify on rich text; React escaping
CSRF	Low	High	SameSite=Strict cookie; custom header check
Broken Auth (brute)	Medium	Critical	Rate limit 5/15min; TOTP MFA; lockout
IDOR (data traversal)	Medium	Critical	clinic_id scope + RBAC + PostgreSQL RLS
Credential leak	Low	Critical	Gitleaks in CI; no secrets in code/logs
Insider threat	Low	High	Audit log every PHI access; anomaly alerts

Data breach (DB dump)	**Low**	**Critical**	AES-256-GCM at app layer; breach ≠ data loss
DoS / DDoS	**Medium**	**Medium**	Cloudflare WAF; rate limiting; auto-scaling
Dependency vulnerability	**High**	**Variable**	Dependabot + npm audit in CI; block critical CVE

09. Data Flow Diagrams

9.1 Medical Record Creation Flow

Figure 3 — Data Flow: Create Medical Record

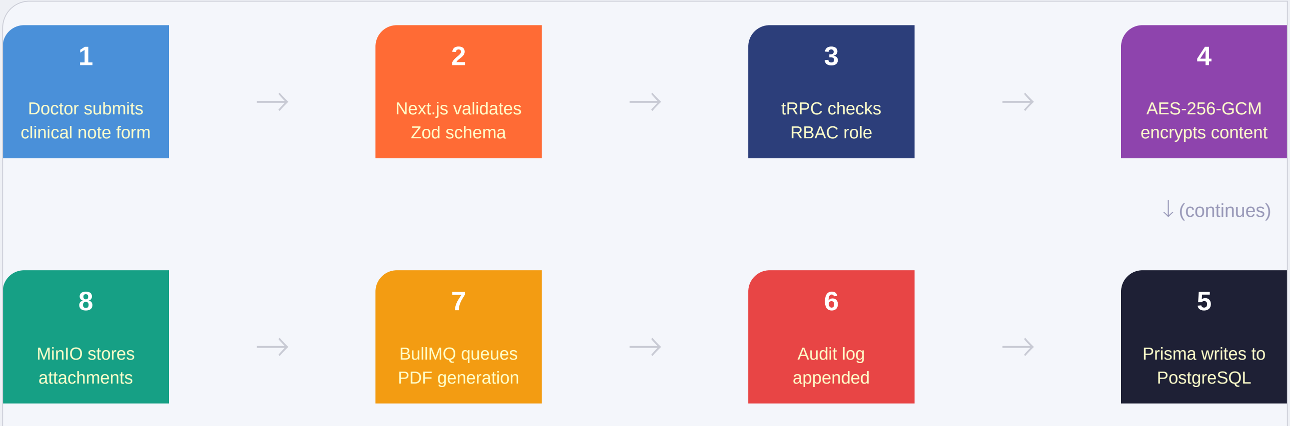

Figure 3 · Data flow through all layers when a doctor creates a medical record

9.2 Authentication Data Flow

Step	From	To	Data	Protocol
1	Browser	Next.js API	POST {email, password}	HTTPS/REST
2	Next.js API	Redis	Check auth:fail:{email}	Redis CMD
3	Next.js API	PostgreSQL	SELECT user WHERE email=?	SQL/Prisma
4	Next.js API	App memory	bcrypt.compare(pw, hash)	In-process
5	Next.js API	Redis	SET rt:{token_hash} userId	Redis CMD
6	Next.js API	Browser cookie	Set-Cookie: rt=... HttpOnly	HTTP header
7	Next.js API	Browser	{accessToken}	JSON body
8	Next.js API	PostgreSQL	INSERT audit_logs (LOGIN)	SQL/Prisma
9	Browser	Next.js tRPC	Authorization: Bearer {at}	HTTPS header
10	Next.js tRPC	App memory	jwt.verify(token)	In-process

9.3 AI Symptom Check Flow

Step	Component	Action	Async?
1	Doctor UI	Submit symptom description + patient context	No
2	tRPC aiRouter	Auth check: role must be DOCTOR	No

3	AI Service	Construct prompt with patient history + symptoms	No
4	Ollama Docker	POST /api/chat with llama3.2 model	No (await)
5	AI Service	Parse JSON response; extract differential diagnoses	No
6	AI Service	Attach confidence scores (logprob conversion)	No
7	tRPC aiRouter	Log AI call to audit_logs (model, prompt hash)	Yes (BullMQ)
8	Doctor UI	Render suggestion cards — NOT auto-applied to record	No
9	Doctor decision	Doctor accepts/rejects each suggestion manually	User action

10. Deployment Architecture

10.1 Infrastructure Overview

Figure 4 — Deployment Topology

Figure 4 · Production deployment topology — all layers from client to observability

10.2 Docker Compose Services

Service Name	Image	Port	Role	Volumes
web	ghcr.io/chr-system/app:latest	3000	Next.js app server	N/A (stateless)
postgres	postgres:16-alpine	5432	Primary database	pg_data:/var/lib/postgresql/data
pgbouncer	edoburu/pgbouncer:latest	5433	Connection pool (100→20)	N/A
redis	redis:7-alpine	6379	Cache, sessions, BullMQ	redis_data:/data
minio	minio/minio:latest	9000/9001	Object storage + web UI	minio_data:/data

meilisearch	getmeili/meilisearch:v1.7	7700	Full-text search engine	meili_data:/meili_data
ollama	ollama/ollama:latest	11434	Local LLM inference server	ollama_data:/root/.ollama
prometheus	prom/prometheus:latest	9090	Metrics scraping + alerting	prom_data:/prometheus
grafana	grafana/grafana:latest	3001	Metrics dashboards	grafana_data:/var/lib/grafana
worker	ghcr.io/chr-system/app:latest	—	BullMQ worker (PDF/email/AI)	N/A (reads env)

10.3 Environment Variables

Variable	Example / Source	Required	Secret?
DATABASE_URL	postgresql://...@postgres	Yes	**Yes**
REDIS_URL	redis://redis:6379	Yes	**No**
NEXTAUTH_SECRET	openssl rand -base64 32	Yes	**Yes**
ACCESS_TOKEN_SECRET	openssl rand -base64 32	Yes	**Yes**
RECORD_ENCRYPTION_KEY	openssl rand -base64 32	Yes	**Yes — rotate quarterly**
MINIO_ROOT_USER	admin	Yes	**Yes**
MINIO_ROOT_PASSWORD	strong-password	Yes	**Yes**
MEILISEARCH_API_KEY	random 64-char string	Yes	**Yes**
OLLAMA_BASE_URL	http://ollama:11434	Yes	**No**
SMTP_HOST	smtp.gmail.com	Yes	**No**
SMTP_PASSWORD	app-specific password	Yes	**Yes**
SENTRY_DSN	https://xxx@sentry.io/n	No	**No**
NEXTAUTH_URL	https://chr.yourdomain.com	Yes	**No**

11. Performance & Scalability Design

11.1 Caching Strategy

Cache Key Pattern	TTL	Data Cached	Invalidation
patient:{id}	5 min	Full patient profile + relations	On patient UPDATE
appointments:doctor:{id}:date	2 min	Day's appointment list	On booking/cancel
vitals:patient:{id}:latest	30 sec	Most recent vitals reading	On new vital INSERT
user:session:{token_hash}	15 min	Authenticated user object	On logout / role change
search:patients:{query_hash}	60 sec	Meilisearch query result page	On patient list change
ai:differential:{hash}	10 min	Ollama response for same input	Manual / TTL only
ratelimit:auth:{email}	15 min	Failed login count	TTL expiry / reset on success
ratelimit:api:{ip}	1 min	Sliding window request count	TTL expiry

11.2 Performance Targets & SLA

Metric	Target	Measurement	Alert Threshold
API response time (p50)	< 50ms	Prometheus histogram	> 100ms
API response time (p95)	< 150ms	Prometheus histogram	> 300ms
API response time (p99)	< 500ms	Prometheus histogram	> 1000ms
Patient search latency	< 100ms	Meilisearch analytics	> 200ms
AI symptom check (p50)	< 3s	BullMQ job duration	> 8s
PDF generation (async)	< 10s	BullMQ job duration	> 30s
DB query time (p95)	< 20ms	pg_stat_statements	> 50ms
Error rate	< 0.1%	Sentry rate dashboard	> 1%
Concurrent users (sustained)	1,000	k6 load test	< 800 under load
Uptime	99.5%	UptimeRobot / Grafana	3 consecutive failures

12. Error Handling & Resilience

12.1 Error Handling Strategy

All errors in CHR-System are handled at the boundary layer (tRPC procedure or API route handler). Internal errors are never surfaced raw to the client. Every error is assigned a unique requestId for cross-system trace correlation.

Error Type	Handling Strategy	Client Response
Validation Error	Zod parse failure — caught in tRPC middleware	422 + field-level error messages
Auth Error	JWT verify throws — caught in auth middleware	401 Unauthorized
RBAC Error	Role check fails — caught in RBAC middleware	403 Forbidden
DB Error (not found)	Prisma P2025 — caught in service layer	404 Not Found
DB Error (conflict)	Prisma P2034 (version mismatch) — optimistic lock	409 Record Locked
DB Error (conn)	Connection pool exhausted — caught in service	503 + Retry-After
Ollama timeout	AbortController 30s — degrade gracefully	200 + {suggestions: [], error: 'AI unavailable'}
MinIO error	Upload fails — job retried 3x via BullMQ	Async: retry; sync: 500 + requestId
Unhandled exception	Global error boundary in Next.js + Sentry capture	500 + {requestId} for support

12.2 Resilience Patterns

Circuit Breaker (Ollama)
If AI fails 3x in 60s → open circuit; return degraded response; check every 30s

Retry with Backoff (BullMQ)
Jobs retry up to 3 times: 1s → 5s → 30s exponential backoff

Connection Pool (PgBouncer)
Max 20 server connections; app pool size 10; queue overflow → 503

Health Checks
GET /api/health returns {db, redis, minio, ollama} status; used by Docker HEALTHCHECK

Graceful Shutdown
SIGTERM → drain in-flight requests → close DB pool → exit; BullMQ completes current job

Idempotency Keys
Client sends X-Idempotency-Key on mutations; server caches response 24h in Redis

13. Logging, Monitoring & Observability

13.1 Structured Logging

All logs are emitted as JSON via Pino (fastest Node.js logger):

```
{
  "level": "info",
  "timestamp": "2026-06-27T09:42:13.000Z",
  "requestId": "uuid-v4",
  "userId": "usr_01...",
  "clinicId": "cln_01...",
  "method": "POST",
  "path": "/api/patients",
  "durationMs": 34,
  "statusCode": 201,
  "ip": "192.168.1.45"
}
```

13.2 Metrics (Prometheus)

Metric Name	Type	Labels	Description
http_request_duration_seconds	Histogram	method, path, status	API latency distribution
http_requests_total	Counter	method, path, status	Total request count
db_query_duration_seconds	Histogram	operation, table	PostgreSQL query latency
redis_operation_duration	Histogram	command	Redis command latency
bullmq_job_duration_seconds	Histogram	queue, status	Background job duration
auth_failures_total	Counter	reason	Login/auth failure count
active_connections	Gauge	service	Current open connections
ai_request_duration_seconds	Histogram	model	Ollama inference latency
audit_log_writes_total	Counter	action, resource	PHI access event count

13.3 Alerting Rules

Alert Name	Condition	Severity	Action
HighErrorRate	error_rate > 1% over 5m	**Critical**	PagerDuty + Slack
SlowAPIResponse	p95 latency > 300ms over 10m	**Warning**	Slack notification
DBConnectionPool	active_connections > 18 sustained 2m	**Warning**	Slack notification
AuthBruteForce	auth_failures > 20 in 1m from same IP	**Critical**	Auto-block IP + alert
AuditLogFailure	audit_log_writes fail 3x	**Critical**	Page on-call engineer

OllamaDown	ai_health = 0 for 5m	**Info**	Log; degrade gracefully
DiskSpaceMinIO	minio_disk_used > 80%	**Warning**	Slack + ticket
HighCPU	CPU > 85% sustained 10m	**Warning**	Scale up notice

14. Testing Architecture

14.1 Test Pyramid

Layer	Tool	Count Target	Speed	Runs On	Coverage Focus
Unit	Vitest	200+ tests	< 5s	Every commit (local)	Business logic, utils, validators
Integration	Vitest + PG	80+ tests	< 60s	PR / CI	tRPC procedures, DB transactions
E2E	Playwright	30+ flows	< 5min	PR to main / nightly	Critical user journeys
Security	OWASP ZAP	Automated	< 10min	PR to main	OWASP Top 10 endpoints
Performance	k6	5 scenarios	< 5min	Pre-release	API SLA targets
Accessibility	axe-core	All pages	< 2min	PR to main	WCAG 2.1 AA
Smoke	Playwright	5 critical	< 30s	Post-deploy	Login, create patient, book

14.2 Key Test Cases

Test ID	Scenario	Expected	Type
TC-001	Login with valid credentials	200 + access token	Integration
TC-002	Login with wrong password 5x	429 + lockout	Integration
TC-003	DOCTOR accesses another clinic's patient	403 CLINIC_SCOPE	Security
TC-004	NURSE attempts to create a medical record	403 FORBIDDEN	Integration
TC-005	Create record — verify DB content encrypted	ciphertext ≠ plaintext	Integration
TC-006	Create record — verify audit_log entry exists	1 row in audit_logs	Integration
TC-007	Concurrent booking — seat double-booking	Only 1 succeeds	Integration
TC-008	SQL injection in patient search param	422 Validation Error	Security
TC-009	XSS payload in clinical notes field	Sanitised output	Security
TC-010	Expired JWT access to protected route	401 Unauthorized	Integration
TC-011	500 concurrent users — p95 response time	< 150ms	Performance
TC-012	E2E: Doctor creates and views patient record	Flow completes < 3s	E2E
TC-013	Patient portal: patient books own appointment	Confirmation email sent	E2E
TC-014	WCAG: All forms navigable by keyboard only	No axe violations	Accessibility

15. Architecture Decision Records (ADR)

ADR-001 — Use tRPC instead of REST-only API

STATUS	ADR-001
ACCEPTED	Architecture Decision Record

Context:

Internal client-server calls benefit from end-to-end type safety. API contract drift between frontend and backend was a common bug source.

Decision:

Use tRPC for all Next.js internal calls. Expose REST wrappers only for third-party integration (webhooks, FHIR export).

Consequences:

- Pro: TypeScript types propagate automatically from server to client
- Pro: No code generation step — types inferred at compile time
- Con: Learning curve for engineers unfamiliar with tRPC
- Con: REST-only consumers need a thin REST adapter layer

ADR-002 — Encrypt medical record content at application layer

STATUS	ADR-002
ACCEPTED	Architecture Decision Record

Context:

Database-level encryption (pgcrypto) protects against disk theft but not against DB admin misuse. PHI requires defence in depth.

Decision:

Encrypt clinical notes with AES-256-GCM before Prisma INSERT. Store IV + auth tag alongside ciphertext. Key held in env var only.

Consequences:

- Pro: DB compromise alone does not expose PHI
- Pro: Meets HIPAA encryption-at-rest requirement
- Con: Cannot do full-text search on encrypted fields (mitigated by Meilisearch on metadata)
- Con: Application holds decryption key — key management must be rigorous

ADR-003 — Use Ollama (local) instead of OpenAI API for AI features

STATUS	ADR-003
ACCEPTED	Architecture Decision Record

Context:

OpenAI API has per-token cost and sends PHI to a third party, violating HIPAA's BAA requirement without a signed agreement.

Decision:

Run Ollama with llama3.2:3b in a Docker sidecar. All inference stays on-premises. HuggingFace free-tier used for specialised models.

Consequences:

- Pro: Zero recurring cost; PHI never leaves the deployment environment
- Pro: No HIPAA BAA required for the AI layer
- Con: llama3.2 quality below GPT-4o — appropriate for decision support, not diagnosis
- Con: Requires GPU/sufficient RAM in production; degrades on small instances

ADR-004 — PostgreSQL + Prisma over NoSQL / Firebase

STATUS	ADR-004
ACCEPTED	Architecture Decision Record

Context:

Firebase Firestore (existing stack) lacks JOIN queries, row-level security, audit capabilities, and SQL-based reporting.

Decision:

Migrate to PostgreSQL 16 with Prisma ORM. Use PgBouncer for connection pooling. Apply RLS policies as secondary RBAC layer.

Consequences:

- Pro: Full SQL power — JOINs, aggregations, CTEs for complex health reports
- Pro: RLS as database-layer defence beyond application RBAC
- Pro: ACID transactions for appointment concurrency
- Con: Migration effort from Firestore; schema design upfront required
- Con: Infrastructure overhead vs Firebase's serverless model

ADR-005 — UUID v7 primary keys over sequential integers

STATUS	ADR-005
ACCEPTED	Architecture Decision Record

Context:

Sequential integer PKs leak record counts to clients and are predictable (enables enumeration attacks via IDOR).

Decision:

Use UUID v7 (time-sortable, RFC 9562) for all primary keys. Generated at application layer by the `uuidv7` npm package.

Consequences:

- Pro: No sequential enumeration possible — IDOR attack surface reduced
- Pro: Time-sortable — preserves insertion order without createdAt index for basic sorts
- Pro: Globally unique — safe for future distributed/sharded architecture
- Con: Slightly larger index size vs integer (16 bytes vs 4 bytes)
- Con: Less human-readable in logs — mitigated by always logging alongside email/MRN

16. Glossary

Term	Definition
Access Token	Short-lived JWT (15 min) used to authenticate API requests
Audit Trail	Append-only log of every PHI access — who, what, when, from where
BullMQ	Redis-backed job queue for async tasks (PDF, email, AI)
C4 Model	Architecture visualisation at Context, Container, Component, Code levels
Circuit Breaker	Pattern that stops calling a failing service to allow recovery
Clinic Scope	Each data row belongs to a clinic; users only see their clinic's data
CSP Nonce	Per-request random token that authorises inline scripts — prevents XSS
FHIR R4	HL7 Fast Healthcare Interoperability Resources — standard health data format
Idempotency Key	Client-provided UUID ensuring duplicate requests produce same result once
ISR	Incremental Static Regeneration — Next.js revalidates static pages on a timer
Keyset Pagination	Cursor-based pagination using a WHERE id > cursor — stable, scalable
MFA (TOTP)	Multi-Factor Auth using Time-based One-Time Passwords (RFC 6238)
MinIO	Open-source S3-compatible object storage for file attachments
Meilisearch	Open-source, ultra-fast full-text search engine (Rust-based)
Ollama	Framework to run LLMs locally — used for private, cost-free AI inference
PgBouncer	PostgreSQL connection pooler — reduces DB server connection overhead
PHI	Protected Health Information — patient data regulated by HIPAA
Prisma	TypeScript ORM with compile-time type safety and migration tooling
RLS	Row-Level Security — PostgreSQL policy enforcing per-row access rules
Refresh Token	Long-lived (7 day) token to obtain new access tokens; stored as bcrypt hash
tRPC	TypeScript RPC framework — zero schema definition, types inferred end-to-end
UUID v7	Time-sortable universally unique identifier (RFC 9562) — used for all PKs
Zod	TypeScript-first schema validation library used for all API input/output